

HARP: Hop-Aware Selective Release for Privacy-Audited GNN Prediction APIs

Krish Sharma

Academy of Math, Science, and Applied Technology
Ed W. Clark High School
Las Vegas, NV, USA
krish.sharma@unlv.edu

Shaikh Arifuzzaman

Department of Computer Science
University of Nevada, Las Vegas
Las Vegas, NV, USA
shaikh.arifuzzaman@unlv.edu

Abstract—Graph neural networks are increasingly served as prediction APIs: clients query a node and receive a softmax score vector used for ranking, routing, and analytics on graphs from thousands to millions of nodes. At this scale the API is a continuous data product—operators must keep accuracy high, keep most responses bit-exact for caching and compliance, and remain *audit-compliant* under membership inference checks. Uniform Laplace posterior perturbation (LBP) cannot meet that joint goal: any $b > 0$ perturbs *every* response, so a clean-majority constraint is infeasible by construction. We formalize this as *constrained score release*—maximize accuracy subject to a membership audit and a minimum exact-response fraction—and propose HARP, a selective release framework for GNN score APIs. HARP is not proposed as a stronger membership defense than uniform noise; it is the mechanism that can satisfy a clean-majority constraint while remaining audit-compliant and recovering accuracy versus published strong LBP. HARP ranks nodes (topology or audit-guided vulnerability), expands high-risk seeds by k hops, and applies a protector only on that set, with Frac/Mass dials and a session budget against re-query averaging. Across six graphs, GAT/inductive checks, ogbn-arxiv (~169k) with defense-aware LiRA, and an ogbn-products subsampled audit (~15k of 2.4M), HARP recovers large accuracy drops versus strong LBP while enabling a 60% clean majority that uniform noise cannot provide.

Index Terms—graph neural networks, prediction APIs, Big Data systems, selective release, calibration, membership inference

I. INTRODUCTION

A. Motivation

Graph neural networks (GNNs) are widely deployed as *prediction APIs*: a client queries a node ID and receives a softmax score vector [9], [10]. Those scores drive ranking, triage, and analytics. They also leak whether a node was in the training set, via membership inference attacks (MIAs) [1], [2], [4], [5].

At Big Data scale the API is not a one-shot model export. It is a stream of released probabilities. Operators therefore face three systems concerns: **volume** (graphs from 10^3 to 10^6 + nodes), **velocity** (clients may re-query randomized answers), and **veracity** (many pipelines need stable, well-calibrated scores for caching, A/B tests, and compliance).

Research conducted with the Data Science Laboratory (DiSC), University of Nevada, Las Vegas.

The usual graph defense is Laplace posterior perturbation (LBP) [5]. Strong uniform noise ($b=0.3$) lowers attack success but often collapses accuracy. Weaker noise keeps accuracy but still perturbs *every* node and raises expected calibration error (ECE). Neither option yields a *clean majority* of exact scores—a concrete data-quality property for downstream consumers.

The systems question is therefore **constrained budget allocation**: maximize task accuracy subject to a membership audit *and* a minimum fraction of bit-exact responses.

a) Main idea.: Membership signal on graphs is not node-local: message passing mixes neighborhoods. HARP (Hop-Aware selective Release Policy) ranks nodes with a cheap topology estimator (LTE) or an audit-guided vulnerability score, expands high-risk seeds by k hops into a protected set $\mathcal{P}_k$, and applies a protector (strong Laplace or selective masking) *only* on that set. Under our locked setting (protected fraction Frac=0.40), about 60% of API responses stay bit-exact—impossible for any uniform $b > 0$.

B. Gap

Prior defenses include LBP [5], MemGuard [6], train-time GNN MIA methods (GTD, MaskArmor) [21], [22], AdvReg [15], and training-time DP-GNNs [13], [14], [27], [36]. MemGuard and uniform LBP both perturb every response; train-time methods do not expose Mass/Frac release accounting. None jointly provide a hop-expanded release set, constructor $\times$ protector policy grid, Mass/Frac dials, matched-budget calibration, and release complexity under standard membership audits. HARP targets that systems gap.

a) Running example.: On Cora ($n=2708$), strong LBP cuts accuracy from ≈ 0.88 to ≈ 0.71 . Weak uniform noise preserves accuracy but perturbs every score (ECE ≈ 0.16). HARP matches that weak-noise budget, keeps $\approx 60\%$ of scores exact, reaches ECE ≈ 0.033 and Acc ≈ 0.83 , and only modestly changes membership-attack AUROC. The operator gains usable accuracy and reproducible scores for most queries—not a claim that clean nodes are “membership-safe.”

b) Non-goal (stated up front).: HARP is *not* a claim of lower population LiRA than equal-mass or strong uniform LBP. When ExactFrac= 0 is allowed, uniform noise is often

the better Acc–LiRA point. HARP’s claim is narrower and systems-shaped: it is the feasible class of policies under $\text{ExactFrac} \geq c > 0$, with hop-consistent allocation, operator di-als, and Acc/calibration recovery versus the published strong-LBP recipe.

C. Contributions and paper map

- 1) **Constrained score release.** Formalize maximize Acc s.t. $\text{LiRA} \leq \tau$ and $\text{ExactFrac} \geq c$; show uniform LBP is infeasible for $c > 0$ (Remark 1); prove hop-consistency is necessary under one-hop score mixing (Prop. 1) and give selective local- ε accounting (Prop. 3).
- 2) **Selective release framework.** Constructor $\times$ protector $\times$ Mass/Frac $\times$ session budget B , with $O(|E| + n \log n)$ setup (Prop. 2).
- 3) **Systems headline.** Versus strong LBP: +12.5 Acc on Cora at $\text{Frac}=0.40$; versus equal-mass: 60% clean majority and $\sim 5\times$ lower ECE; quantified cache-hit simulation for ExactFrac ; clean-slice fidelity across datasets.
- 4) **Scale audits.** ogbn-arxiv Acc recovery with defense-aware LiRA ($n_{\text{sh}} \in \{2, 4\}$); ogbn-products systems probe plus subsampled LiRA on a 15k-node induced sub-graph [28].

Section II related work; Section III threat model; Section IV HARP; Sections VI–VII evaluation; Section VIII discussion; Section IX conclusion.

II. BACKGROUND AND RELATED WORK

A. Membership inference on classical models

Shadow-model membership inference [1] and simple confidence features [2] show that prediction APIs leak training membership [3], [33]. Related extraction and inversion attacks [25], [26] further motivate treating scores as a privacy surface. Likelihood-ratio attacks (LiRA) are the current audit standard [4]. We use LiRA as an *operator audit* of whether release noise is warranted—not as the sole paper objective. We also track calibration because overconfident scores are a recurring membership cue [17], [34].

B. Release-time defenses on scores

MemGuard [6] adversarially perturbs confidence vectors to confuse attack classifiers while preserving the top-1 label; prediction purification similarly sanitizes released scores. Both act on *every* response and therefore cannot satisfy a clean-majority constraint. LBP [5] is the graph-native uniform Laplace baseline. HARP differs by selecting a hop-consistent minority and leaving the rest bit-exact.

C. Membership inference on graphs

Olatunji et al. [5] demonstrated node-level black-box MIAs on GNNs and proposed LBP and neighborhood sampling defenses. He et al. [38] systematically study node-level MIAs under varying topology knowledge. Subsequent work studies graph-level membership [7], embedding leakage [24], and broader attack taxonomies [8]. GTD [21] targets the

transductive setting with alternate training and loss flattening. Khosla [39] shows that training-graph construction and inference-time edge access directly modulate membership advantage—orthogonal to our score-API audit protocol. GPS [35] studies structure $\rightarrow$ attribute leakage; we target score-based *membership*. Our threat model matches the score-based black-box node setting; we do not claim white-box, structural-modification, or federated attack coverage.

D. Architectures, regularization, and privacy

GNNs [9] apply spectral neighborhood averaging; GraphSAGE [10] uses mean aggregation and is often more robust under heterophily. DropEdge [11] randomly deletes edges during training. DP-SGD and DP-GNNs [13], [14] provide formal privacy guarantees at training time. Homophily modulates the train/test ϕ -gap [18]–[20]: under low h and sparsity, supervised memorization widens membership cues—motivating risk-weighted budget allocation.

E. Positioning

Our contribution is a **selective release framework** for GNN score APIs with Mass/Frac systems metrics. Uniform output noise (LBP, MemGuard) and train-time defenses (GTD, MaskArmor) are baselines. Formal DP (GAP) remains the right tool when an ε guarantee is required [13]; we do not re-implement GAP under our splits. In short: LBP/MemGuard spend on every node; GTD/MaskArmor harden training without release Mass/Frac; GAP answers formal ε ; HARP answers constrained exact-fraction release under audits.

a) *Systems lens (why Big Data).*: Training systems optimize memory and throughput. Score-API release is an adjacent surface: every prediction is a data product with a utility and quality cost. HARP logs Frac and Mass the way operators already log batch size. We compare against the published strong LBP recipe ($b=0.3$) and against equal-mass weak noise so reviewers see what the same budget buys when spent uniformly.

III. PROBLEM SETUP

A. Notation

Let $G = (V, E)$ be an undirected graph with $n = |V|$ nodes, feature matrix $X \in \mathbb{R}^{n \times d}$, and labels $y \in \{0, \dots, C-1\}^n$. A random split partitions V into supervised train $\mathcal{T}$, validation $\mathcal{V}$, and test $\mathcal{Q}$ with ratios 40%/20%/40% (resampled per seed). Training is *transductive*: every node participates in message passing; only $\mathcal{T}$ enters the cross-entropy loss. We write $\hat{p}_v$ for the softmax posterior at v , r_v for a risk score in $[0, 1]$, $\mathcal{P}_k$ for the hop-expanded protected set, $\text{Mass} = \sum_v \sigma_v$, and $\text{Frac} = |\mathcal{P}_k|/n$.

B. Constrained score release

Operators rarely optimize Acc alone. They need usable accuracy *and* a reproducible majority of API responses, subject to a membership audit.

Definition 1 (Constrained Score Release). *Given a trained GNN and release channel, choose a release policy Π to*

$$\max_{\Pi} \text{Acc}(\Pi) \quad \text{s.t.} \quad \text{LiRA}(\Pi) \leq \tau, \quad \text{ExactFrac}(\Pi) \geq c,$$

where ExactFrac is the fraction of nodes whose released posterior equals $\hat{p}_v$ bit-exactly, $\tau \in [0.5, 1]$ is an audit threshold, and $c \in [0, 1]$ is the required clean fraction.

Remark 1 (Uniform LBP is infeasible for $c > 0$). *Independent Laplace noise with scale $b > 0$ on every coordinate changes the value almost surely, so $\text{ExactFrac} = 0$ for any uniform LBP policy. Selective release with protected fraction Frac achieves $\text{ExactFrac} = 1 - \text{Frac}$ by construction.*

This reframes the Acc–LiRA comparison. Equal-mass LBP can win Acc subject to $\text{LiRA} \leq \tau$ when $c=0$ is allowed; HARP solves the strictly harder problem with $c > 0$. The Frac sweep traces HARP’s Pareto frontier under that constraint.

C. Threat model

The adversary is a black-box client: it queries softmax scores and tries to guess whether a node was in the training set [2], [4], [5]. It does not see gradients or embeddings. White-box, link-stealing, and label-only APIs are out of scope [8], [12].

We also stress three practical variants: defense-aware shadows, multi-query averaging ($K \in \{1, 5, 20\}$), and an LTE-aware query adversary that focuses on clean nodes, protected-set boundaries, and top-decile LTE seeds (Sec. VII-C0k). We do not claim robustness to poisoning or unbounded queries without a session budget.

D. Attack features (brief)

Following Salem-style score attacks [2], we use confidence, true-class probability, entropy, and a margin term as ϕ -features. These are the signals membership audits and optional train-time alignment target.

E. Metrics

Utility: test accuracy (primary), plus F1/AUROC where useful. **Membership audit:** we report LiRA AUROC [4] and TPR at 0.1%/1% FPR—how well an attacker separates train members from held-out nodes (0.5 = chance). We also report confidence-based attacks for continuity with prior GNN work [2], [5]. **Data quality:** ECE [17], clean-slice ECE, and the clean-score majority ($1 - \text{Frac}$). LiRA tells operators *whether* to spend noise; Acc and ECE tell them *what they paid* in utility and score fidelity.

Remark 2. *The primary protocol is transductive (non-members remain in the graph during training), matching common semi-supervised GNN APIs. We additionally report an inductive edge-exclusion check on Cora (Sec. VII-C0l) where test-incident edges are dropped at train/infer time.*

HARP: Hop-Aware Risk-conditioned Privacy



Fig. 1. HARP pipeline: risk ranking $\rightarrow$ high-risk seeds $\rightarrow$ k -hop expansion $\rightarrow$ protector only on the protected set.

IV. HARP: HOP-AWARE SELECTIVE RELEASE

A. Design principle

Uniform posterior noise wastes budget on low-risk nodes and cannot meet $\text{ExactFrac} \geq c > 0$. Node-local risk weighting is still incomplete for GNNs: if v is protected but a neighbor u is released cleanly, aggregation can reintroduce membership signal into u ’s score. HARP therefore builds a hop-consistent protected set inside a small policy grid (Fig. 1):

- 1) **Constructor:** rank nodes with LTE (topology), audit-guided vulnerability, or random risk.
- 2) Pick high-risk seeds (or search the seed fraction to hit a target Frac).
- 3) Expand $\mathcal{P}_k = \text{Ball}_k(\mathcal{R})$ on the undirected graph.
- 4) **Protector:** strong Laplace on $\mathcal{P}_k$, or selective masking (exact on clean; one-hot top-1 on $\mathcal{P}_k$).
- 5) Optionally train with risk-weighted ϕ -alignment on the same set; enforce session budget B at serving time.

Locked HARP uses LTE + Laplace at $\text{Frac}=0.40$; audit seeds and masking are framework ablations.

a) *Noise mass and clean fraction.*: $\text{Mass} = \sum_v \sigma_v$ and $\text{Frac} = |\mathcal{P}_k|/n$ are operator accounting metrics. Uniform LBP at scale b has $\text{Mass}=bn$ and $\text{Frac}=1$. See paragraph IV-D0a for the locked-budget identity.

b) *Locked-config example (Cora).*: On Cora ($n=2708$), $\text{Frac}=0.40$ protects ≈ 1083 nodes after one-hop expansion; ≈ 1625 nodes stay exact. $\text{Mass}_{\text{HARP}} \approx 325$ matches equal-mass LBP with $b=0.12$. Strong LBP ($b=0.3$) spends $\text{Mass} \approx 812$ on all nodes—the Acc-collapse baseline in Table I.

c) *Release transform (Laplace protector).*: Let $\sigma_v = \sigma_{\text{strong}} \mathbb{1}[v \in \mathcal{P}_k]$. The released posterior is

$$\tilde{p}_v = \text{renorm}([\hat{p}_v + \text{Lap}(0, \sigma_v)]_+), \quad (1)$$

with independent Laplace coordinates and the same transform on every defense-aware shadow. The masking protector instead returns $\hat{p}_v$ for $v \notin \mathcal{P}_k$ and a one-hot of $\arg \max \hat{p}_v$ for $v \in \mathcal{P}_k$.

d) *Algorithm (systems view).*:

- 1) **Risk:** $r \leftarrow \text{Constructor}(G, \mathcal{T})$ in $O(|E|)$ (LTE) or via a small shadow ensemble (audit).
- 2) **Seeds:** binary-search seed fraction so $|\text{Ball}_k(\mathcal{R})|/n \approx \text{Frac}^*$.
- 3) **Expand:** $\mathcal{P}_k \leftarrow \text{Ball}_k(\mathcal{R})$ via BFS ($O(|E|)$).
- 4) **Train (optional):** CE + risk-weighted AdvReg/MMD with $r_v \cdot \mathbb{1}[v \in \mathcal{P}_k]$.

- 5) **Release:** apply (1) (or the masking protector); log (Mass, Frac, Acc, LiRA, ECE).

B. Risk constructors

LTE uses only G , $y|_{\mathcal{T}}$, and the train mask:

$$s_v^{\text{deg}} = \frac{1}{1+\deg(v)}, \quad s_v^{\text{het}} = 1 - \hat{h}_v, \quad s_v^{\text{sup}} = \frac{|\mathcal{N}(v) \cap \mathcal{T}|}{\max(1, \deg(v))}, \quad (2)$$

with $\hat{h}_v$ the train-neighbor label agreement. Raw r_v is the mean of the three terms, min-max normalized. LTE is a cheap correlative proxy—not an oracle that predicts population LiRA alone.

Audit-guided ranking (HARP-audit) reuses the shadow ensemble already needed for LiRA: for each node, score $|\mu_{\text{IN}} - \mu_{\text{OUT}}|$ of logit-true-class confidence across shadows (Aerni-style vulnerability [37]), then seed and hop-expand at matched Frac. When shadows are affordable, audit ranking is the preferred constructor; LTE remains the no-shadow approximation.

Random ranking is the matched-Frac control isolating topology/audit signal from selectivity itself. Empirically, LTE beats random on LiRA for heterophilic Chameleon/Actor (Table IX); audit ranking targets the Cora case where LTE loses LiRA. Frac/Mass remain the primary population levers; the constructor is a ranking prior inside a fixed budget.

C. Why k -hop expansion

Let $\mathcal{R}$ be seeds. A GNN score at u already mixes information from $\text{Ball}_k(u)$ before any release noise is applied. If a membership-sensitive seed $v \in \mathcal{R}$ is noised at release but a neighbor u is returned bit-exact, the adversary simply queries u .

Proposition 1 (Hop-consistency necessity under one-hop mixing). *Fix a one-hop score model $s_u = \alpha z_v + (1 - \alpha) z_u$ for $u \in \mathcal{N}(v)$, $\alpha \in (0, 1]$, where z_v carries a membership cue of gap $\gamma > 0$ between member and non-member worlds, and release adds independent Laplace noise only to chosen nodes' scores. If the release policy noises v but leaves u exact, then any adversary that observes s_u retains advantage equal to the unprotected leaf advantage. Noising $\text{Ball}_1(\{v\})$ is necessary and sufficient to place Laplace noise on every score that linearly depends on z_v under this mixer.*

Proof sketch. Under seed-only protection the released leaf score equals the clean mixer output, so the member/non-member gap in $\mathbb{E}[s_u]$ remains $\alpha\gamma$. Expanding protection to u adds $\text{Lap}(0, \sigma)$ to s_u , which does not change the gap in expectation but is the unique one-hop cover that subjects every α -dependent coordinate to noise. A synthetic mixer confirms the ordering: unprotected and seed-only leaf AUROCs stay near 1.0, while hop-1 protection collapses the leaf attack (Fig. 5, left). $\square$

Frac—not k —is the population Acc–Mass knob; k is a coverage constraint on protected-set geometry. We keep $k=1$ as the default for 2-layer SAGE/GCN; $k \in \{0, 2\}$ appear in the consolidated ablation (Table IX).

D. Complexity and accounting

Proposition 2 (HARP setup and release complexity). *Let $G = (V, E)$ have $n = |V|$ nodes and $m = |E|$ adjacency endpoints, with C classes. Fix hop radius $k = O(1)$ and $T = O(1)$ binary-search iterations for a target protected fraction. Then setup (LTE + seeds + k -hop expand) runs in $O(m + n \log n)$ time and $O(n + m)$ space; release runs in $O(nC)$ —matching uniform LBP's tier, with less noise-sampling work whenever $\text{Frac} < 1$.*

Proof sketch. LTE scans edges then normalizes n scores: $O(m+n)$. Seed selection is $O(n \log n)$ plus T BFS expansions at $O(m+n)$. Release writes scales, samples Laplace on $\mathcal{P}_k$, and renormalizes: $O(nC)$. $\square$

Proposition 3 (Selective local privacy accounting). *Interpret each node's clean posterior as a query with ℓ_1 -sensitivity at most $\Delta=2$ (worst-case change on the simplex). Independent $\text{Lap}(\sigma)$ on every coordinate of $v \in \mathcal{P}_k$ yields $\varepsilon_{\text{loc}} = \Delta/\sigma$ local DP for that node's release; nodes in $V \setminus \mathcal{P}_k$ have $\varepsilon=\infty$. Consequently any policy with $\text{ExactFrac} > 0$ has infinite global ε — $\text{ExactFrac} > 0$ and global DP are incompatible by construction.*

At the locked point ($\sigma=0.30$, $\text{Frac}=0.40$) this gives $\varepsilon_{\text{loc}}=2/0.3 \approx 6.67$ on the protected minority and $\varepsilon=\infty$ on the 60% clean majority. This is the constrained-release tradeoff stated formally: HARP provides a finite local guarantee where it spends noise, and an explicit non-guarantee where it keeps scores exact. When a legal global ε is required, use training-time DP (e.g., GAP) instead [13].

a) *Accounting identity (stated once).*: Under locked Frac and fixed σ_{strong} , $\text{Mass}_{\text{HARP}} = \text{Frac} \cdot n \cdot \sigma_{\text{strong}}$ by construction—an accounting identity, not an empirical finding. The empirical contributions are (i) Acc recovery at fixed strong σ versus published LBP $b=0.3$, and (ii) calibration and ExactFrac at matched Mass versus weak uniform noise. We do not restate this caveat elsewhere.

E. What HARP is not

HARP is not differential privacy [27] and is not a claim of lower population LiRA than uniform noise under $\text{ExactFrac}=0$. It is an empirical selective release framework for the constrained problem: spend a protector on a hop-consistent minority, keep the rest exact, and audit with membership attacks.

a) *Intuition.*: Think of HARP as a two-tier API. Low-risk nodes return the model's exact softmax. High-risk nodes (and their one-hop neighbors) get strong Laplace—or label-only masking—on a minority of the graph. The constructor ranks *where* to spend; Frac sets *how much*; the session budget limits re-query averaging.

V. MOTIVATION: WHEN SELECTIVE PRIVACY PAYS

Matched-budget undefended LiRA (Table XII) concentrates leakage in small/mid heterophilic cells; several mid/large graphs sit near chance under $n_{\text{shadows}}=4$. That audit motivates selective spend: do not pay strong uniform noise

where LiRA $\approx$ 0.5; deploy HARP where LiRA $\gg$ 0.5 or where veracity (exact/calibrated scores) is required under a strong- σ policy.

a) *Three deployment regimes.*: Heterophilic graphs (Chameleon/Actor) show auditable LiRA and LTE helps at fixed Frac; homophilic Cora/Citeseer emphasize veracity at matched Mass; large/audit-null graphs (PubMed/Photo/ogbn) often warrant Frac=0 unless policy forces strong σ (ogbn Acc recovery +0.420).

VI. EXPERIMENTAL PROTOCOL

A. Pre-declared rules

Primary privacy = LiRA AUROC; primary utility = test Acc; systems = noise mass, protected fraction, and clean-score majority (1 – Frac). Hyperparameters are not selected to minimize test LiRA. All HARP comparisons against LBP use the published strong scale $b=0.3$ as the primary baseline; equal-mass $b=0.120$ is a mandatory fairness control, not the headline recipe.

B. Systems metrics (definitions)

We log four operator-facing quantities alongside Acc and LiRA:

- **Mass** = $\sum_v \sigma_v$: total Laplace scale spent at release (auditable noise budget).
- **Frac** = $|\mathcal{P}_k|/n$: fraction of nodes that receive strong noise.
- **Clean majority** = $1 - \text{Frac}$: share of API responses returned bit-exact.
- **ECE**: whether released probabilities match empirical frequencies (lower is better calibrated).

Frac is the main dial: at fixed strong scale σ_{strong} , increasing Frac spends more Mass and perturbs more nodes, usually improving LiRA but hurting Acc and reducing the clean majority (Table VI). These metrics are cheap to log alongside Acc/LiRA and make privacy spend auditable the way train-seconds and peak RSS make training spend auditable.

C. Locked HARP

$\lambda=0.5$, $k=1$, $\sigma_{\text{strong}}=0.30$, target Frac=0.40, HCAG on for GraphSAGE/GCN, train-on-protected, warmup 5 (Cora val Acc; not test LiRA). Baselines use LBP $b=0.3$ / equal-mass $b=0.120$; GTD $\gamma=1.0$; MaskArmor top- k masking; MemGuard max- $\ell_1=0.2$; DP-SGD Acc-tuned (vacuous ϵ); HARP $k \in \{0, 1, 2\}$, audit/random seeds, and selective masking at the same Frac. GAT runs use the same Frac/ σ lock with HCAG off.

D. Baselines and attacks

none; strong LBP; equal-mass LBP; MemGuard [6]; GTD [21]; MaskArmor [22]; clip+noise DP-SGD; HARP and hop/Frac/constructor/protector ablations. Gaussian LiRA with $n_{\text{shadows}}=4$ (primary); we also sweep $n_{\text{shadows}} \in \{4, 16, 64\}$ on Cora/Chameleon for stability. Defense-aware shadows share the release transform. Five seeds with paired bootstrap CIs on Cora fairness/Frac; three seeds elsewhere unless

noted. ogbn-arxiv uses NeighborLoader, $n_{\text{shadows}}=2$, gate off, warmup 3 (cost); Frac/ σ remain locked.

E. What each baseline tests

We include defenses that isolate different design choices rather than maximize row count. **Strong LBP** ($b=0.3$) is the published recipe operators would copy; it anchors the Acc-collapse problem. **Equal-mass LBP** ($b=0.12$) matches HARP’s Mass at Frac=0.40 and answers: “what if we spent the same budget uniformly on every node?” **MemGuard** is a simplified MemGuard-style baseline (random-search perturbation preserving argmax; not a full adversarial-training reproduction of Jia et al.): it perturbs every score, so ExactFrac=0—the natural competitor when $c=0$ is allowed. **Selective masking** is the simpler protector alternative (exact on clean; label-only on $\mathcal{P}_k$). **GTD** and **MaskArmor** are train-time GNN MIA defenses; they harden training rather than add release-time Laplace, so Mass/Frac are not applicable (Table I, “—”). **Clip+noise DP-SGD** provides a formal-privacy anchor (with vacuous ϵ at our Acc-tuned point). **HARP ablations** (k , release-only, random/audit seeds, masking) attribute Acc and LiRA effects to hop expansion, training alignment, constructor, and protector respectively.

F. Attack protocol (summary)

We train defense-aware shadow models, release transformed posteriors, and report LiRA AUROC with ϕ -features [2], [4]. Validation nodes are excluded from membership labels; test Acc/ECE use the held-out test split. Multi-query experiments average $K \in \{1, 5, 20\}$ independent draws; session policies cap fresh draws at $B=5$.

G. Negative controls

PubMed (near-chance undefended LiRA): selective privacy should not be forced to spend a full uniform budget; HARP still preserves Acc vs. strong LBP at Frac=0.40 (Acc 0.839 vs. 0.768). ogbn-arxiv / ogbn-products: near-chance or memory-bounded audits under matched budgets—audit first, harden second [28].

VII. RESULTS

A. Overview

We organize results around operator questions: **(Q1)** Can HARP keep accuracy and score quality at a matched noise budget? **(Q2)** Does it recover Acc versus strong uniform LBP? **(Q3)** How do Frac, multi-query, and session budgets behave? **(Q4)** Do patterns hold across architectures, inductive splits, and larger OGB graphs? Equal-mass LBP remains the honest Acc–membership baseline when exact scores are not required.

B. How to read the trade space

Three columns answer different goals. **Acc** is task utility. **LiRA** is a membership audit (lower is harder to attack). **ECE** / **clean majority** is data quality for downstream consumers. Equal-mass LBP and MemGuard can look competitive on Acc–LiRA while still noising every response (ExactFrac=0).

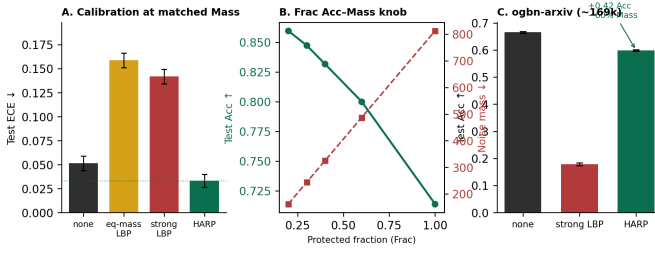


Fig. 2. Systems headline. (A) ECE at matched Mass. (B) Frac Acc–Mass dial. (C) ogbn-arxiv Acc vs. strong LBP.

TABLE I

CORA GRAPH-SAGE SYSTEMS COMPARISON (5 SEEDS). HEADLINE AT MATCHED MASS $\approx$ 325: HARP ECE 0.033 [0.028, 0.038] VS. EQUAL-MASS 0.159 [0.153, 0.164]; Δ ACC VS. STRONG LBP +0.125 [0.113, 0.136]. HARP LiRA TPR@1% FPR 0.011 (NONE 0.033). [†]CLIP+NOISE DP-SGD: ACC-TUNED POINT WITH VACUOUS $\varepsilon \approx 10^{-3}$ (NOT GAP). [‡]GTD/MASKARMOR: TRAIN-TIME DEFENSES ONLY—NO RELEASE LAPLACE BUDGET, SO MASS/FRAC ARE N/A (— —). [§]MEMGUARD PERTURBS EVERY RESPONSE (EXACTFRAC=0); MASS IS ℓ_1 PERTURBATION MASS.

Defense	Acc	LiRA	ECE $\downarrow$	Mass	Frac
none	0.877	0.567	0.051	—	—
LBP $b=0.3$	0.707	0.517	0.142	812	1.00
LBP equal-mass	0.870	0.520	0.159	325	1.00
MemGuard [§] [6]	0.879	0.538	0.122	—	1.00
GTD [‡] [21]	0.878	0.538	0.091	—	—
MaskArmor [‡] [22]	0.877	0.580	0.123	—	—
DP-SGD [†]	0.784	0.509	—	—	1.00
HARP	0.832	0.533	0.033	325	0.40
HARP-mask	0.885	0.639	0.092	0	0.40

HARP’s headline is Acc recovery plus exact/calibrated scores under $\text{ExactFrac} \geq c$ (Remark 1). Pick the constraint first—especially whether $c > 0$ is required—then read Tables I–IV.

C. HARP systems headline: accuracy and score quality at matched Mass

a) *Primary claim.*: Fig. 2 and Table I: at matched Mass $\approx$ 325 on Cora, equal-mass weak LBP ($b=0.120$) perturbs every node (ECE 0.159). HARP yields ECE 0.033 with Frac=0.40—nearly $5\times$ better calibration—and a 60% bit-exact clean majority. Equal-mass cannot produce exact scores.

b) *Utility vs. published strong LBP.*: Against LBP $b=0.3$ [5], HARP raises Acc 0.707 $\rightarrow$ 0.832 (Δ Acc +0.125) at LiRA 0.517 $\rightarrow$ 0.533 and Frac=0.40 (Tables I, III). The operational win is accuracy at the same strong per-node noise on a minority of nodes. The pattern repeats on Citeseer/Chameleon/Actor/PubMed/Photo (Table III).

c) *Membership audit (honest secondary).*: At matched Mass, weak uniform LBP can beat HARP on Acc–LiRA (Cora Acc 0.870 / LiRA 0.520). If the only goal is Acc subject to $\text{LiRA} \leq \tau$ and exact scores do not matter, prefer equal-mass LBP. HARP is for operators who also need $\text{ExactFrac} \geq c > 0$ —not a stronger membership defense than uniform noise.

GTD and MaskArmor rows report Acc/LiRA/ECE only: both defenses modify training, not release posteriors, so they have no Mass or Frac to compare against HARP’s release

TABLE II
CLEAN-SLICE ECE UNDER LOCKED HARP (3 SEEDS; CORA ALIGNS WITH THE 5-SEED SLICE STUDY).

Dataset	ECE _{clean}	ECE _{prot}	ECE _{pop}	none ECE
Cora	0.020	0.090	0.037	0.053
Citeseer	0.083	0.121	0.094	0.140
Chameleon	0.273	0.186	0.238	0.290
Actor	0.254	0.277	0.262	0.262

TABLE III

Δ ACC (HARP–STRONG LBP) UNDER LOCKED FRAC=0.40.

Dataset	Δ Acc	Mass _{LBP}	Mass _{HARP}	Frac
Cora	+0.125	812	325	0.40
Citeseer	+0.096	998	399	0.40
Chameleon	+0.061	683	273	0.40
Actor	+0.040	2280	912	0.40
PubMed	+0.070	5915	2366	0.40
Photo	+0.185	2295	946	0.40

TABLE IV

MATCHED-MASS: EQUAL-MASS LBP ($b=0.12$) VS. HARP (3 SEEDS; CORA: 5).

Dataset	HARP Acc	Eq Acc	HARP ECE	Eq ECE	Clean?
Cora	0.832	0.870	0.033	0.159	HARP 60%
Citeseer	0.716	0.740	0.094	0.073	HARP 60%
Chameleon	0.452	0.439	0.238	0.214	HARP 60%
Actor	0.327	0.324	0.262	0.244	HARP 60%

budget. MemGuard preserves Acc but leaves $\text{ExactFrac} = 0$ and only modestly reduces LiRA under our audit. Selective masking keeps $\text{ExactFrac} = 0.60$ and Acc high, but raises LiRA (clean scores still leak; one-hot protected scores remain confident)—justifying the Laplace protector when the audit requires it.

Slice ECE localizes the win on Cora (5 seeds): $\text{ECE}_{\text{clean}} = 0.018$ vs. $\text{ECE}_{\text{prot}} = 0.081$ (population 0.033). Clean nodes match undefended calibration; protected nodes carry the noise penalty. HARP’s population ECE can beat undefended none because train-time ϕ -alignment/HGAG reduces overconfidence on the clean majority; on Chameleon, HARP Acc can slightly exceed undefended Acc for the same reason. Across datasets, the portable quality property is the clean majority and clean-slice ECE (Table II). Equal-mass population ECE can look competitive when weak noise regularizes overconfidence (Table IV), but every response is still perturbed.

d) *Multi-dataset grid.*: Table V repeats the Acc/LiRA pattern on six graphs; PubMed/Photo are near-chance LiRA controls where HARP still preserves Acc vs. strong LBP. Reading down each triple: HARP consistently recovers 4–19 Acc points vs. strong LBP while LiRA moves modestly (often +0.01–+0.02 vs. strong LBP, still below undefended). At the security-relevant low-FPR point, HARP also cuts TPR@1% FPR relative to undefended (Cora 0.033 $\rightarrow$ 0.011; Chameleon 0.059 $\rightarrow$ 0.011; Citeseer 0.042 $\rightarrow$ 0.014). The heterophilic rows (Chameleon, Actor) show the largest undefended LiRA

TABLE V
HARP PRIMARY GRID (GRAPHSAGE; CORA 5-SEED, OTHERS 3-SEED).
TPR = LiRA TPR@1% FPR.

Dataset	Defense	Acc	LiRA	TPR	Mass	Frac
Cora	none	0.877	0.567	0.033	—	—
	LBP $b=0.3$	0.707	0.517	0.019	812	1.00
	HARP	0.832	0.533	0.011	325	0.40
Citeseer	none	0.744	0.591	0.042	—	—
	LBP $b=0.3$	0.620	0.523	0.030	998	1.00
	HARP	0.716	0.548	0.014	399	0.40
Chameleon	none	0.448	0.618	0.059	—	—
	LBP $b=0.3$	0.391	0.539	0.038	683	1.00
	HARP	0.452	0.555	0.011	273	0.40
Actor	none	0.335	0.610	0.037	—	—
	LBP $b=0.3$	0.287	0.549	0.032	2280	1.00
	HARP	0.327	0.561	0.019	912	0.40
PubMed	none	0.876	0.505	0.012	—	—
	LBP $b=0.3$	0.768	0.499	0.010	5915	1.00
	HARP	0.839	0.502	0.012	2366	0.40
Photo	none	0.802	0.500	—	—	—
	LBP $b=0.3$	0.552	0.499	—	2295	1.00
	HARP	0.737	0.501	—	946	0.40

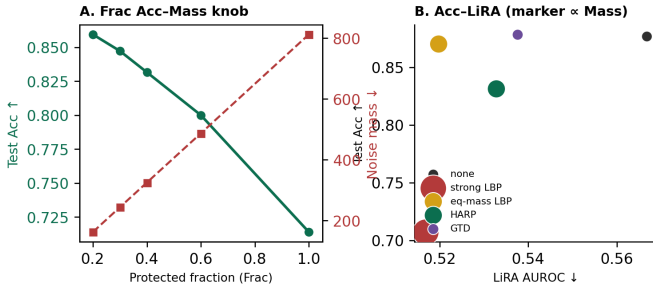


Fig. 3. Frac dial (left) and Acc–LiRA tradeoff with marker size $\propto$ Mass (right). Equal-mass LBP and GTD shown for context.

TABLE VI
HARP FRAC SWEEP ON CORA (5 SEEDS; $\sigma_{\text{strong}}=0.30$).

Frac	Acc $\uparrow$	LiRA $\downarrow$	Mass $\downarrow$
0.20	0.859 [0.855, 0.866]	0.537 [0.532, 0.543]	162
0.30	0.847 [0.843, 0.853]	0.538 [0.533, 0.543]	244
0.40	0.832 [0.826, 0.839]	0.533 [0.525, 0.541]	325
0.60	0.800 [0.793, 0.809]	0.519 [0.512, 0.526]	487
1.00	0.714 [0.699, 0.727]	0.511 [0.503, 0.519]	812

and the clearest case for spending a Frac budget; the near-chance rows (PubMed, Photo) show that HARP’s Acc benefit does not require a population LiRA win—operators audit first, then decide whether Frac >0 is warranted.

e) *Frac dial and Mass accounting.*: Fig. 3 and Table VI sweep Frac on Cora (5 seeds): Acc falls and LiRA improves as Frac rises. Frac is the Acc–Mass dial under fixed σ_{strong} —the systems analog of an empirical privacy spend knob.

f) *Exact scores $\neq$ membership safety.*: Clean nodes are well calibrated ($\text{ECE}_{\text{clean}}=0.018$), but they are not automatically hard to attack. Under locked HARP, unprotected-slice confidence AUROC is ≈ 0.64 vs. ≈ 0.51 population-wide (Fig. 4; Table XI). Raise Frac or enforce session budget B if clean-slice canaries drift up. Aerni et al. [37] motivate such canary checks (Table XIV(a)).

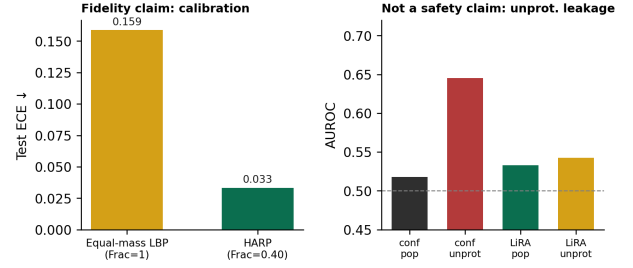


Fig. 4. Veracity vs. safety on Cora. Left: ECE at matched Mass. Right: unprotected-slice confidence AUROC remains 0.645.

TABLE VII
MULTI-QUERY LiRA ON CORA GRAPHSAGE (3 SEEDS; $n_{\text{shadows}}=4$).

Defense	K=1 LiRA	K=5 LiRA	K=20 LiRA
none	0.568	0.568	0.568
HARP	0.534	0.546	0.550
eq-mass LBP	0.520	0.520	0.520

TABLE VIII
SESSION BUDGET ON HARP CORA AT $K=20$ (3 SEEDS). $B=1$ FREEZES ACC/ECE NEAR THE ONE-SHOT RELEASE; $B=20$ MATCHES UNCAPPED AVERAGING.

Policy	B	Acc	LiRA
Budget reuse	1	0.836	0.544
Budget reuse	5	0.847	0.544
Budget reuse	20	0.886	0.548
Uncapped avg	—	0.886	0.548

g) *Velocity: multi-query and session budgets.*: Averaging K independent releases denoises Laplace noise and moves the API toward the undefended baseline (Table VII). On Cora, HARP LiRA rises from 0.534 at $K=1$ to 0.550 at $K=20$, while HARP accuracy rises from 0.836 to 0.886; equal-mass LiRA stays at 0.520 because every node is always noised. A session policy of at most B fresh draws per node—reusing the last draw thereafter—caps this drift (Table VIII). Sweeping $B \in \{1, 5, 20\}$ at $K=20$ shows $B=1$ freezes the one-shot defense (recommended default when velocity is adversarial); $B=5$ is a mild relaxation for benign re-queries. HARP without a session budget is a one-shot defense; Frac and B are complementary velocity controls.

h) *Component ablations (consolidated).*: Table IX answers “what matters?” in one place. Release-only HARP recovers most Acc vs. strong LBP—selectivity carries the utility win; $k \in \{0, 1, 2\}$ barely moves population LiRA at matched Frac (hop geometry, not a population lever). Constructors: LTE beats random seeds on LiRA for heterophilic Chameleon/Actor; on Cora, random is slightly better on LiRA while LTE gains Acc; audit-guided ranking sits between and is preferred only when the LiRA shadow ensemble is already paid for. GAT matches the Acc–Mass pattern (Cora Acc 0.692 $\rightarrow$ 0.816).

i) *Why ExactFrac matters (cache simulation).*: A Cora-sized synthetic API (800 clients $\times$ 25 re-queries; exact-match

TABLE IX

CONSOLIDATED ABLATIONS AT MATCHED FRAC $\approx$ 0.40 (3 SEEDS). TOP: CORA VARIANTS. BOTTOM: CONSTRUCTOR ACC/LiRA (LTE / RANDOM / AUDIT).

Cora variant	Acc	LiRA	Mass	Train s
LBP $b=0.3$	0.716	0.517	812	0.8
HARP (locked, $k=1$)	0.836	0.534	325	2.5
HARP $k=0$ / $k=2$	0.834 / 0.828	0.524 / 0.529	325 / 334	2.4
HARP release-only	0.830	0.539	325	0.9
HARP random seeds	0.826	0.526	325	2.0
HARP (GAT) [29]	0.816	0.525	325	1.0
Constructor	Acc / LiRA at Frac=0.40			
	LTE	Random	Audit	
Cora	0.836 / 0.534	0.818 / 0.522	0.830 / 0.528	
Citeseer	0.716 / 0.548	0.700 / 0.540	0.715 / 0.540	
Chameleon	0.452 / 0.555	0.467 / 0.556	0.463 / 0.561	
Actor	0.327 / 0.562	0.319 / 0.565	0.326 / 0.562	

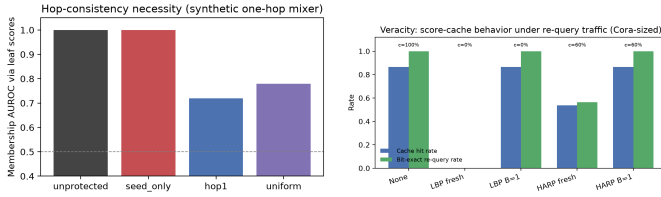


Fig. 5. Left: hop necessity (Prop. 1)—seed-only leaves leaf AUROC at 1.0; hop-1 collapses it. Right: score-cache hits under re-query traffic (Table X).

TABLE X

SCORE-CACHE SIMULATION (CORA-SIZED; EXACTFRAC c IS BY DESIGN).

Policy	c	Hit rate	Bit-exact re-query
None	1.00	0.865	1.000
LBP fresh	0.00	0.000	0.000
LBP $B=1$	0.00	0.865	1.000
HARP fresh	0.60	0.537	0.563
HARP $B=1$	0.60	0.865	1.000

score cache) makes veracity concrete (Fig. 5, right; Table X): fresh LBP hits 0%; HARP hits $\approx$ 54% from ExactFrac alone; $B=1$ freezes both, but only HARP keeps $c=0.60$ for consumers needing bit-identical softmax vectors.

j) *Qualitative failure case (Chameleon)*.: Population metrics hide mechanism. Of 375 protected train nodes (seed 42), only 13 ($\approx$ 3.5%) failed to shrink their train–test true-class confidence gap under HARP—overall mean gap falls $0.46 \rightarrow 0.11$. Manual inspection of those 12 failures shows a recurring pattern: Lap($\sigma=0.3$) plus renormalization can increase true-class confidence on some draws (e.g., node 177: $0.78 \rightarrow 1.00$; node 381: $0.34 \rightarrow 0.81$), concentrating mass on the true class by chance. Peak clean posteriors (conf > 0.9) are also hard to flatten. The takeaway for operators: HARP reliably softens the mid-confidence mass that drives population LiRA, but a small overconfident/unlucky tail remains—raise Frac or σ on that slice if canaries demand it, rather than expecting uniform-LBP-level erasure at ExactFrac=0.60.

k) *Adaptive LTE-aware queries*.: Table XI: restricting queries to unprotected nodes, protected-set boundary cleans, or top-decile LTE seeds raises conf AUROC $0.513 \rightarrow 0.638$ —

TABLE XI

LTE-AWARE ADAPTIVE QUERIES ON CORA HARP (3 SEEDS).

Query set	Conf@0.4	Conf@0.6	LiRA@0.4
Population	0.513	0.471	0.534
Unprotected (clean)	0.638	0.632	0.539
Boundary (clean)	0.626	0.634	0.535
Top-decile LTE	0.651	0.505	0.532

TABLE XII

AUDIT CONTROLS (GRAPHSAGE; 3 SEEDS). *HETEROPHILIC. (B) COMPARATIVE SHADOW SCALING (NOT HARP-ONLY).

(a) Undefended LiRA ($n_{sh}=4$)			(b) Cora shadow scaling (LiRA / TPR@1%)			
Dataset	n	Acc	LiRA	none	LBP	HARP
Chameleon*	2 277	0.449	0.623	0.568 / 0.033	0.517 / 0.022	0.534 / 0.014
Cora	2 708	0.879	0.668	0.808 / 0.201	0.555 / 0.008	0.670 / 0.057
Actor*	7 600	0.335	0.610	0.847 / 0.265	0.561 / 0.008	0.677 / 0.075
Photo	7 650	0.825	0.500			
PubMed	19 717	0.876	0.505			
ogbn-arxiv	169 343	0.666	0.505			

As invariant: none 0.879, LBP 0.716, HARP 0.836. Chameleon mirrors the pattern (at $n_{sh}=64$: none 0.965, LBP 0.685, HARP 0.811; Δ Acc vs. LBP +0.061).

0.651 under locked HARP (3 seeds), while subset LiRA stays near population ($\approx$ 0.53–0.54). Population LiRA understates conf-MIA on the clean majority—log unprot./boundary canaries and raise Frac or B if they drift. Raising Frac to 0.60 collapses the top-decile LTE canary ($0.651 \rightarrow 0.505$) while residual clean-slice conf AUROC remains elevated ($\approx$ 0.63)—pair Frac escalation with session budget $B=1$.

l) *Inductive edge exclusion*.: Dropping test-incident edges (Cora; 3 seeds) raises undefended LiRA to 0.631 (TPR@1% FPR 0.036). HARP recovers Acc $0.486 \rightarrow 0.620$ vs. strong LBP at Frac=0.40 (LiRA 0.569; ECE 0.037). At Frac=0.60, Acc falls to 0.549 while LiRA improves to 0.559—inductive audits may warrant a higher Frac on the same dial.

D. When to spend: volume $\times$ structure

Table XII(a): undefended LiRA concentrates in small/mid heterophilic cells; Photo/PubMed/ogbn-arxiv sit near chance under matched shadows—audit first, then set Frac. Table XII(b) reports the honest shadow-count story on Cora for none / LBP / HARP. Absolute LiRA rises with $n_{shadows}$ for every defense (Gaussian LiRA sharpens [4]); Acc is invariant. **HARP does not beat strong LBP on LiRA at large shadow counts**—at $n_{sh}=64$ on Cora, LBP LiRA 0.561 vs. HARP 0.677—expected when ExactFrac=0 is allowed. What HARP keeps is (i) strictly lower LiRA than undefended and (ii) large Acc recovery vs. strong LBP (Δ Acc +0.12, invariant in n_{sh}). TPR@1% FPR follows the same pattern.

E. Canaries and large-graph volume

Table XIV(a): HARP suppresses planted/top-decile confidence cues relative to none/LBP; population LiRA remains primary. Table XIV(b): on ogbn-arxiv ($\sim$ 169k; Neighbor-Loader; gate off, warmup 3) [28], strong LBP collapses Acc to 0.179 [0.175, 0.184] while HARP recovers 0.599 [0.597, 0.601] (paired Δ Acc +0.420 [0.414, 0.425]; 3 seeds)

TABLE XIII
OGBN-PRODUCTS: SYSTEMS PROBE (FULL GRAPH) AND BFS-INDUCED
15K-NODE LiRA AUDIT (829K EDGES; 3 SEEDS; $n_{sh}=2$).

Setting	n	Acc	LiRA	Mass / RSS
Load graph	2.45M	—	—	RSS ≈ 6.2 GB
Train none (subsample Acc)	2.45M	0.695	—	RSS ≈ 7.4 GB
Strong LBP budget	2.45M	—	—	Mass $\approx 7.3 \times 10^5$
HARP budget (Frac=0.40)	2.45M	—	—	Mass $\approx 2.9 \times 10^5$
BFS-sub none	15k	0.874	0.506	—
BFS-sub LBP $b=0.3$	15k	0.343	0.500	—
BFS-sub HARP	15k	0.686	0.503	—

TABLE XIV
CANARIES (LEFT) AND OGBN-ARXIV VOLUME (RIGHT; 3 SEEDS). LiRA
SHOWN FOR $n_{sh}=2$ (PRIMARY) AND $n_{sh}=4$ (STRESS).

(a) Cora canaries					(b) ogbn-arxiv		
Def.	LiRA	Top	Plt.	ECE	Acc	LiRA ₂	LiRA ₄
none	0.568	0.551	0.527	0.055	0.666	0.502	0.503
LBP	0.517	0.548	0.543	0.145	0.179	0.498	0.501
HARP	0.534	0.266	0.423	0.087	0.599	0.514	0.505

at Frac=0.40. Defense-aware LiRA stays near chance at both $n_{sh}=2$ and $n_{sh}=4$ (Table XIV(b))—a volume Acc-recovery result under an audit-null regime, not a claim that ogbn needs Frac>0. On ogbn-products (~2.4M nodes) [28], Table XIII reports the systems probe on a 16 GB host plus a defense-aware LiRA audit on a BFS-induced 15k-node subgraph (829k edges; $n_{sh}=2$). HARP recovers Acc 0.343 $\rightarrow$ 0.686 vs. strong LBP while LiRA stays near chance—the same Acc-recovery pattern as ogbn-arxiv, now with an honest large-graph audit. Full-graph products LiRA exceeds host memory; the BFS subsample closes that gap without inventing a full-graph privacy claim. The Big Data boundary: release accounting and loader serving scale to millions of nodes; full-graph shadow audits need more RAM or distributed training.

If audit LiRA $\approx$ 0.5, prefer Frac=0; if LiRA $\gg$ 0.5 or veracity requires ExactFrac $\geq c$, set Frac on val Acc (Frac=0.2 maximizes Acc on Cora/Chameleon; Frac=0.4 is the locked compliance default with 60% exact responses).

a) *Systems microbenchmarks.*: Cora LTE+hop setup $\approx$ 41 ms; HARP release $\approx$ 0.51 ms vs. LBP $\approx$ 0.71 ms; products serving $\approx 3.3 \times 10^3$ QPS at ≈ 7.4 GB RSS.

b) *Operator workflow.*: Audit with defense-aware LiRA; log Mass/Frac/ECE/unprot-conf. If LiRA $\approx$ 0.5, set Frac=0; else select Frac on validation Acc (Table VI), enforce session budget $B=1$ by default (Table VIII), and use GAP when a legal ε is required [13]. On Cora/Chameleon, val Acc selection recovers the Acc-maximizing Frac in the grid (Frac=0.2 matches the test oracle); LiRA or ExactFrac constraints instead pick a higher Frac on the same Pareto frontier. Re-audit after model or graph updates.

F. Protocol limitations

Equal-mass and MemGuard can beat HARP on Acc–LiRA when ExactFrac= 0 is allowed; selective masking raises LiRA; multi-query needs $B=1$ for a one-shot guarantee; adaptive

conf AUROC remains elevated on residual clean nodes even at Frac=0.60 (Table XI); HARP is not DP; ogbn-products full LiRA needs more RAM/distributed shadows.

VIII. DISCUSSION

A. What HARP is for

HARP is for teams that serve GNN scores as a data product and need three things at once: usable accuracy, a clean majority of reproducible probabilities (ExactFrac $\geq c$), and an auditable noise budget. It is *not* a drop-in replacement for formal differential privacy: when a legal ε is required, use training-time DP such as GAP [13], [16]. Membership audits (LiRA) tell the operator whether Frac should be zero or positive; Acc and ECE tell them what the chosen Frac costs. When LiRA $\approx$ 0.5, the systems recommendation is Frac=0. When LiRA still mandates strong σ or ExactFrac $\geq c>0$, selective release is the feasible class; uniform LBP is not (Remark 1).

B. Impact for Big Data graph APIs

Hop-consistent release sets are the analogue of topology-aware row budgets; Mass/Frac give spend telemetry. ExactFrac enables score-cache hits that uniform noise cannot (Table X); heterophilic graphs recover Acc vs. $b=0.3$; audit-null large graphs argue against blanket noise.

C. Threats to validity

Cora fairness/Frac: five seeds with paired CIs; other cells: three seeds. Mass/Frac are spend metrics, not (ε, δ) ; ExactFrac/ECE measure fidelity, not per-node hardness. Products subsample LiRA and ogbn-arxiv $n_{sh}=4$ extend scale audits without claiming full-graph products privacy.

D. Relation to formal differential privacy

Prop. 3: finite local ε on the protected minority, $\varepsilon=\infty$ on the clean majority—hence infinite global ε whenever ExactFrac > 0 . That incompatibility is the constrained-release tradeoff. Use GAP for regulatory DP; use HARP when ExactFrac $\geq c$ matters.

E. What we do not claim

HARP is not a stronger membership defense than uniform noise; does not dominate equal-mass/MemGuard when ExactFrac= 0; hop k is not a monotone LiRA lever; multi-query without B erodes the one-shot defense; clean-slice conf-MIA stays elevated unless Frac or B rises (Table XI).

F. Future work

Adaptive Frac from streaming audits, co-optimizing formal ε with Mass [31], [32], federated hop-respecting APIs, and distributed LiRA at products/papers100M scale.

IX. CONCLUSION

We proposed HARP, a selective release framework for GNN prediction APIs under constrained score release: maximize accuracy subject to a membership audit and a minimum exact-response fraction. HARP is not a stronger membership defense than uniform noise; it is the feasible mechanism under $\text{ExactFrac} \geq c > 0$, with hop-consistent allocation (Prop. 1), selective local- ε accounting (Prop. 3), and Acc/calibration recovery versus strong LBP—including a quantified cache-hit benefit from the clean majority. On six benchmarks plus OGB scale-ups with defense-aware LiRA, membership audits guide *when* to spend budget; Frac and B are the operator dials. Use training-time DP when a legal global ε is required.

a) Reproducibility:

Code

(`defenses/harp.py`, `defenses/memguard.py`, `run_harp_rejection_fix.py`), locked configs, and frozen CSVs accompany the submission artifact at <https://github.com/krishharma/privacy-gnn>. Runs used Darwin arm64 CPU; LiRA $n_{\text{shadows}}=4$ primary (16/64 comparative sweeps on Cora/Chameleon).

REFERENCES

- [1] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2017, pp. 3–18.
- [2] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, “ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models,” in *Proc. NDSS*, 2019.
- [3] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, “Privacy risk in machine learning: Analyzing the connection to overfitting,” in *Proc. IEEE CSF*, 2018, pp. 268–282.
- [4] N. Carlini, S. Chien, M. Nasr, A. Song, T. Terzis, and F. Tramèr, “Membership inference attacks from first principles,” in *Proc. IEEE S&P*, 2022, pp. 1897–1914.
- [5] I. E. Olatunji, W. Nejdl, and M. Khosla, “Membership inference attack on graph neural networks,” arXiv:2101.06570, 2021.
- [6] J. Jia, A. Salem, M. Backes, Y. Zhang, and N. Z. Gong, “MemGuard: Defending against black-box membership inference attacks via adversarial examples,” in *Proc. ACM CCS*, 2019, pp. 259–274.
- [7] Y. Wu, D. Li, B. Chen, and L. Shen, “Adapting membership inference attacks to GNN for graph classification,” in *Proc. IEEE ICDM*, 2021, pp. 1429–1434.
- [8] Y. Zhang, Y. Zhao, Z. Li, X. Cheng, Y. Wang, O. Kotevska, P. S. Yu, and T. Derr, “A survey on privacy in graph neural networks: Attacks, preservation, and applications,” arXiv:2308.16375, 2023.
- [9] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.
- [10] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. NeurIPS*, 2017.
- [11] Y. Rong, W. Huang, T. Xu, and J. Huang, “DropEdge: Towards deep graph convolutional networks on node classification,” in *Proc. ICLR*, 2020.
- [12] C. A. Choquette-Choo, F. Tramèr, N. Carlini, and N. Papernot, “Label-only membership inference attacks,” in *Proc. ICML*, 2021, pp. 1964–1974.
- [13] S. Sajadmanesh et al., “GAP: Differentially private graph neural networks with aggregation perturbation,” in *Proc. USENIX Security*, 2023, pp. 3229–3246.
- [14] M. Abadi et al., “Deep learning with differential privacy,” in *Proc. ACM CCS*, 2016, pp. 308–318.
- [15] M. Nasr, R. Shokri, and A. Houmansadr, “Machine learning with membership privacy using adversarial regularization,” in *Proc. ACM CCS*, 2018, pp. 634–646.
- [16] M. Nasr, R. Shokri, and A. Houmansadr, “Comprehensive privacy analysis of deep learning,” in *Proc. IEEE S&P*, 2019, pp. 739–753.
- [17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.
- [18] J. Zhu et al., “Beyond homophily in graph neural networks: Current limitations and effective designs,” in *Proc. NeurIPS*, 2020.
- [19] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, “Geom-GCN: Geometric graph convolutional networks,” in *Proc. ICLR*, 2020.
- [20] D. Lim, F. Hohne, X. Li, S. L. Huang, V. Gupta, O. Bhalerao, and S. N. Lim, “Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods,” in *Proc. NeurIPS*, 2021.
- [21] P. Niu, C. Pan, S. Chen, and O. Milenković, “Graph Transductive Defense: a Two-Stage Defense for Graph Membership Inference Attacks,” arXiv:2406.07917, 2024.
- [22] C. Chen, X. Zhang, H. Qiu, J. Lou, Z. Liu, and X. Chen, “MaskArmor: Confidence masking-based defense mechanism for GNN against MIA,” *Information Sciences*, vol. 669, 2024.
- [23] M. A. P. Chamikara et al., “Can diffusion-based posterior smoothing secure GNNs from membership inference attacks?” in *Proc. AICCSA*, 2025.
- [24] V. Duddu, A. Boutet, and V. Shejwalkar, “Quantifying privacy leakage in graph embedding,” in *Proc. WSDM*, 2020, pp. 76–84.
- [25] M. Fredrikson, S. Jha, and T. Ristenpart, “Model inversion attacks that exploit confidence information,” in *Proc. ACM CCS*, 2015, pp. 1322–1333.
- [26] F. Tramèr et al., “Stealing machine learning models via prediction APIs,” in *Proc. USENIX Security*, 2016, pp. 601–618.
- [27] C. Dwork, “Differential privacy,” in *Proc. ICALP*, 2006, pp. 1–12.
- [28] W. Hu et al., “Open graph benchmark: Datasets for machine learning on graphs,” in *Proc. NeurIPS*, 2020.
- [29] P. Veličković et al., “Graph attention networks,” in *Proc. ICLR*, 2018.
- [30] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proc. ICLR*, 2019.
- [31] I. Mironov, “Rényi differential privacy,” in *Proc. IEEE CSF*, 2017, pp. 263–275.
- [32] B. Jayaraman and D. Evans, “Evaluating differentially private machine learning in practice,” in *Proc. USENIX Security*, 2019, pp. 1895–1912.
- [33] C. Song, T. Ristenpart, and V. Shmatikov, “Machine learning models that remember too much,” in *Proc. ACM CCS*, 2017, pp. 587–601.
- [34] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. ICML*, 2005, pp. 625–632.
- [35] Y. Jiang et al., “Unveiling privacy vulnerabilities: Investigating the role of structure in graph data,” in *Proc. ACM KDD*, 2024. (GPS: structure→attribute leakage; GHRatio: learnable edge sampling.)
- [36] Q. Yuan et al., “Node-Aware Differentially Private Graph Neural Networks for Graph Representation Learning,” arXiv:2308.04943, 2023.
- [37] M. Aerni, J. Zhang, and F. Tramèr, “Evaluations of machine learning privacy defenses are misleading,” in *Proc. ACM CCS*, 2024.
- [38] X. He, R. Wen, Y. Wu, M. Backes, Y. Shen, and Y. Zhang, “Node-level membership inference attacks against graph neural networks,” arXiv:2102.05429, 2021.
- [39] M. Khosla, “Impact of graph structure on membership-inference risk for graph neural networks,” *Proc. Priv. Enhancing Technol. (PoPETs)*, 2026.